Data Bank SQL Case Study

-Rohith Vaddepally

Introduction: The Data Bank Case study explores how a digital bank manages customer data and financial transactions using a distributed node-based system. The objective of this case study is to analyse customers who are allocated across different nodes and regions, evaluate customer transactions, and provide data allocation strategies for different options.

In this case study, I applied complex SQL techniques, window functions, aggregate functions, and time calculations to investigate how customers are allocated to different nodes, calculate balances, and work on allocation challenges. This will help the Data Bank team to optimise their architecture, improve resource planning, and evaluate different strategies for allocating data storage efficiently.

Schema: The schema has three tables: regions, customer_nodes, and customer_transactions.

Schema available here: [data bank schema]

- The regions tables contain region_id and their respective region_name values.
- Customers are randomly distributed across the nodes according to their region. These nodes change frequently to reduce the risk of hackers gaining access to the Data Bank's system and stealing customers' money and data. Sample data is given below.
- The customer transactions table stores all customer deposits, withdrawals, and purchases made using their Data Bank debit card.

region_id	region_name
1	Africa
2	America
3	Asia
4	Europe
5	Oceania

customer_id	region_id	node_id	start_date	end_date
1	3	4	2020-01-02	2020-01-03
2	3	5	2020-01-03	2020-01-17
3	5	4	2020-01-27	2020-02-18
4	5	4	2020-01-07	2020-01-19
5	3	3	2020-01-15	2020-01-23
6	1	1	2020-01-11	2020-02-06
7	2	5	2020-01-20	2020-02-04
8	1	2	2020-01-15	2020-01-28
9	4	5	2020-01-21	2020-01-25
10	3	4	2020-01-13	2020-01-14

customer_id	txn_date	txn_type	txn_amount
429	2020-01-21	deposit	82
155	2020-01-10	deposit	712
398	2020-01-01	deposit	196
255	2020-01-14	deposit	563
185	2020-01-29	deposit	626
309	2020-01-13	deposit	995
312	2020-01-20	deposit	485
376	2020-01-03	deposit	706
188	2020-01-13	deposit	601
138	2020-01-11	deposit	520

Case Study Questions:

This case study is divided into 4 different parts:

1. Customer nodes exploration.
2. Customer transactions.
3. Data allocation challenge.
4. Extra Challenge.

Customer Nodes Exploration

1. **How many unique nods are there on the Data Bank system?**

```
SELECT COUNT(distinct node_id) AS number_of_unique_nodes
FROM customer_nodes;
```

There are five unique nodes.

2. **What is the number of nodes per region?**

```
SELECT r.region_name,
       COUNT(node_id) AS node_cnt
FROM customer_nodes cn
LEFT JOIN regions r
     ON cn.region_id = r.region_id
GROUP BY r.region_name;
```

Output:

region_name	node_cnt
Africa	714
Europe	616
Australia	770
America	735
Asia	665

3. **How many customers are allocated to each region?**

```
SELECT r.region_name,
       COUNT(Distinct customer_id) AS customer_cnt
FROM customer_nodes cn
LEFT JOIN regions r
     ON cn.region_id = r.region_id
GROUP BY r.region_name;
```

Output:

region_name	customer_cnt
Africa	102
America	105
Asia	95
Australia	110
Europe	88

4. **How many days on average are customers reallocated to a different node?**

```sql
WITH corrected_cust_nodes AS (
SELECT customer_id,
       region_id,
       node_id,
       start_date,
      (CASE
          WHEN end_date = '9999-12-31' THEN '2020-12-31'
          ELSE end_date
      END) AS modified_end_date
FROM customer_nodes
),
```

CTE for correcting the end_date where it is '9999-12-31' to '2020-12-31'.

```sql
diff_in_days AS (
SELECT *,
       datediff(modified_end_date,start_date) AS diff_in_days
FROM corrected_cust_nodes
)

SELECT
       ROUND(AVG(diff_in_days)) AS avg_rellocation_days
FROM diff_in_days;
```

CTE for the difference between the end date and the start date.

It takes averagely 49 days to reallocate the customers to a different node.

5. **What are the median, 80th, and 95th percentiles of the reallocation days metric for each region?**

Reallocation days are the difference in days between when a customer is moved to a different node.

```sql
WITH corrected_cust_nodes AS (
SELECT customer_id,
       region_id,
       node_id,
       start_date,
      (CASE
          WHEN end_date = '9999-12-31' THEN '2020-12-31'
          ELSE end_date
      END) AS modified_end_date
FROM customer_nodes
),
```

CTE corrected_cust_nodes is for modified the end_date in the customer nodes table from 9999-12-31 to 2020-12-31. It creates a new modified_end_date column.

```sql
diff_in_days AS (
SELECT *,
       (datediff(modified_end_date,start_date)) AS diff_in_days
FROM corrected_cust_nodes
),
```

CTE diff_in_days is to find the difference in days b/w start_date and modified_end_date. It creates a new column diff_in_days.

Output till now:

customer_id	region_id	node_id	start_date	modified_end_date	diff_in_days
1	3	4	2020-01-02	2020-01-03	1
2	3	5	2020-01-03	2020-01-17	14
3	5	4	2020-01-27	2020-02-18	22
4	5	4	2020-01-07	2020-01-19	12
5	3	3	2020-01-15	2020-01-23	8
6	1	1	2020-01-11	2020-02-06	26
7	2	5	2020-01-20	2020-02-04	15
8	1	2	2020-01-15	2020-01-28	13
9	4	5	2020-01-21	2020-01-25	4
10	3	4	2020-01-13	2020-01-14	1
11	2	5	2020-01-19	2020-01-25	6

```sql
ranked_regions AS(
SELECT region_id,
       diff_in_days,
       ROW_NUMBER() OVER (PARTITION BY region_id
                          ORDER BY diff_in_days
                          ) AS rn,
       COUNT(*) OVER(PARTITION BY region_id) AS total_rows
FROM diff_in_days
)
```

The CTE ranked_regions is for ranking each region based on the diff_in_days column. Also used the COUNT window function for counting the number of rows for each region.

Region 1- 770 rows, Region 2-735 rows, Region 3-714 rows, Region 4-665 rows, Region 5-616 rows

Output till now:

region_id	diff_in_days	rn	total_rows
1	0	1	770
1	0	2	770
1	0	3	770
1	0	4	770
1	0	5	770
1	0	6	770
1	0	7	770
1	0	8	770
1	0	9	770
1	0	10	770

```
SELECT region_id,
       MAX(CASE WHEN rn=CEIL(total_rows*0.50) THEN diff_in_days END) AS 50_percentile,
       MAX(CASE WHEN rn=CEIL(total_rows*0.80) THEN diff_in_days END) AS 80_percentile,
       MAX(CASE WHEN rn=CEIL(total_rows*0.95) THEN diff_in_days END) AS 95_percentile
FROM ranked_regions
GROUP BY region_id
```

This is the main query to find the median, 80th, and 95th percentiles grouped by each region.

Output:

region_id	50_percentile	80_percentile	95_percentile
1	17	28	265
2	18	27	266
3	17	27	267
4	17	27	269
5	18	28	258

So, for region 4, 50% of customers were reallocated in 17 days, 80% of the customers were reallocated within 27 days, and only 20% of customers took more than 27 days, and 95% of the customers took 269 days, and 5% took more than 269 days.

For region 4, the 80th percentile is calculated by:

= total rows * 0.80
= 665 * 0.80
= 532

532 (rn) is equal to 27 (diff_in_days).

4	27	532	665

CEIL() function: CEIL function returns the smallest integer value that is greater than or equal to a given numeric expression. Ceil() is used in case there are any decimal values.

MAX() function: In the CASE result, all the rows become NULL except the row that has 27 diff_in_days, and MAX() is used to retrieve that row.

Customer Transactions

1. **What is the unique count and total amount for each transaction type?**

```
SELECT txn_type,
       COUNT(*) AS unique_txn_count,
       CONCAT('$',SUM(txn_amount)) AS txn_amount
FROM customer_transactions
GROUP BY txn_type;
```

Output:

txn_type	unique_txn_count	txn_amount
deposit	2671	$1359168
withdrawal	1580	$793003
purchase	1617	$806537

Note: As each row already represents one transaction, so COUNT(*) would work. Using COUNT(txn_type) doesn't work as it counts only rows where txn_type is not null.

2. **What is the average total historical deposit count and amounts for all customers?**

```
WITH deposit_stats AS (
SELECT customer_id,
       COUNT(*) AS deposit_counts,
       SUM(txn_amount) AS deposit_amount
FROM customer_transactions
WHERE txn_type = "deposit"
GROUP BY customer_id
)

SELECT FLOOR(AVG(deposit_counts)) AS avg_number_of_deposits,
       CONCAT("$", CEIL(AVG(deposit_amount))) AS avg_deposited_amount
FROM deposit_stats;
```

Output:

avg_number_of_deposits	avg_deposited_amount
5	$2719

FLOOR() function: The floor() returns the largest integer value that is less than or equal to a given numeric expression. It effectively rounds a number down to the nearest whole number.

3. **For each month, how many Data Bank customers make more than 1 deposit and either purchase or 1 withdrawal in a single month?**

```
WITH txn_type AS (
SELECT DATE_FORMAT(txn_date,'%Y-%m') AS Month,
        customer_id,
       (CASE WHEN txn_type = "deposit" THEN 1 ELSE 0 END) AS deposit_type,
       (CASE WHEN txn_type = "purchase" THEN 1 ELSE 0 END) AS purchase_type,
       (CASE WHEN txn_type = "withdrawal" THEN 1 ELSE 0 END) AS withdrawal_type
FROM customer_transactions
),
```

CTE for creating columns for each txn_type using CASE conditions statements.

```
summing_txn AS (
SELECT Month,
       (customer_id),
       SUM(deposit_type) as deposit_type,
       SUM(purchase_type) as purchase_type,
       SUM(withdrawal_type) as withdrawal_type
FROM txn_type
GROUP BY Month, customer_id
HAVING deposit_type > 1 AND (purchase_type >=1 OR withdrawal_type>=1)
)
```

CTE for summing the totals of different txn_type and aggregating them by month, and customer_id. Also, later filter them using the HAVING clause with the required conditions.

```
SELECT Month,
       COUNT(customer_id) AS customer_cnt
FROM summing_txn
GROUP BY Month
ORDER BY Month;
```

Output:

Month	customer_cnt
2020-01	168
2020-02	181
2020-03	192
2020-04	70

4. **What is the closing balance for each customer at the end of the month?**

```sql
WITH balances AS (
SELECT customer_id,
       txn_date,
       DATE_FORMAT(txn_date,'%Y-%m') AS month,
       SUM(CASE WHEN txn_type = 'deposit' THEN txn_amount ELSE - txn_amount END)
           OVER(PARTITION BY customer_id
           ORDER BY txn_date) AS running_balance
FROM customer_transactions
),
```

CTE for calculating the running balance of the transactions based on the conditions where the deposit is '+' and purchase & withdrawal is '-'.

```sql
ranking_month_end AS (
SELECT customer_id,
       txn_date,
       month,
       running_balance,
       ROW_NUMBER() OVER (PARTITION BY customer_id, month
                          ORDER BY txn_date DESC) AS rn
FROM balances
)
```

CTE for ranking the customers partitioned by each month and in descending txn_date. Window function used is ROW_NUMBER()

```sql
SELECT customer_id,
       month,
       running_balance AS closing_balance
FROM ranking_month_end
where rn=1
ORDER BY customer_id, month;
```

5. **What is the percentage of customers who increase their closing balance by more than 5%?**

This is an extension question to the previous question.

```sql
WITH balances AS (
SELECT customer_id,
       txn_date,
       DATE_FORMAT(txn_date,'%Y-%m') AS month,
       SUM(CASE WHEN txn_type = 'deposit' THEN txn_amount ELSE - txn_amount END)
           OVER(PARTITION BY customer_id
                ORDER BY txn_date) AS running_balance
FROM customer_transactions
),

ranking_month_end AS (
SELECT customer_id,
       txn_date,
       month,
       running_balance,
       ROW_NUMBER() OVER (PARTITION BY customer_id, month
                          ORDER BY txn_date DESC) AS rn
FROM balances
),
```

```sql
closing_balance AS (
SELECT customer_id,
       month,
       running_balance AS closing_balance
FROM ranking_month_end
where rn=1
),

previous_month_bal AS (
SELECT *,
        LAG(closing_balance) OVER(PARTITION BY customer_id
                   ORDER BY month) AS previous_month_closing_balance
FROM closing_balance
),

increase_by AS (
SELECT *,
       ROUND((closing_balance - previous_month_closing_balance)/previous_month_closing_balance, 2) * 100  AS pcnt_increase
FROM previous_month_bal
WHERE previous_month_closing_balance IS NOT NULL
      AND closing_balance > previous_month_closing_balance
      AND ((closing_balance - previous_month_closing_balance)/previous_month_closing_balance) * 100 > 5
)

SELECT ROUND(COUNT(DISTINCT customer_id) * 100 / (SELECT COUNT(DISTINCT customer_id)
                                    FROM customer_transactions)) AS pcnt_of_customers_with_5pcnt_increase
FROM increase_by;
```

After calculating the closing balance for each customer, I used the LAG() function to create a new column, previous_month_closing_balance in the CTE previous_month_bal.

In the CTE increase_by, the percentage increase is calculated with the formula: (closing_balance - previous_month_closing_balance) /previous_month_closing_balance

Condition 1: previous_month_closing_balance is not null (filters only non-null rows, occurred due to LAG() function.

Condition 2: filters rows only where closing_balance is more than previous_month_closing_balance

Condition 3: filters rows only where percentage_increase in more than 5%.

Final query: To find the % of customers who increased their closing balance by 5%. Need to apply the below formula

$$\frac{\text{Count of distinct customers from CTE increase_by}}{\text{Count of distinct customers from the CTE customer_transactions}} * 100$$

Output: 37% of the customers increased their closing balance by 5%.

Data Allocation Challenge

To test out a few different hypotheses - the Data Bank team wants to run an experiment where different groups of customers would be allocated data using 3 different options:

- Option 1: data is allocated based off the amount of money at the end of the previous month
- Option 2: data is allocated on the average amount of money kept in the account in the previous 30 days
- Option 3: data is updated real-time

For this multi-part challenge question - you have been requested to generate the following data elements to help the Data Bank team estimate how much data will need to be provisioned for each option:

- running customer balance column that includes the impact each transaction
- customer balance at the end of each month
- minimum, average and maximum values of the running balance for each customer

Using all of the data available - how much data would have been required for each option on a monthly basis?

```sql
WITH running_balances AS (
SELECT customer_id,
       txn_date,
       MONTHNAME(txn_date) AS month,
       SUM(CASE WHEN txn_type = "deposit" THEN txn_amount ELSE -txn_amount END)
           OVER(PARTITION BY customer_id
                ORDER BY txn_date) AS running_balance
FROM customer_transactions
),

ranking_closing_bal AS (
SELECT *
FROM (
    SELECT *,
            ROW_NUMBER() OVER (PARTITION BY customer_id, month
                               ORDER BY txn_date DESC) AS rn
    FROM running_balances
) rcb
WHERE rn = 1
),
```

```sql
running_bal_stats AS (
SELECT customer_id,
       month,
       ROUND(AVG(running_balance)) AS avg_bal,
       MIN(running_balance) AS min_bal,
       MAX(running_balance) AS max_bal
FROM running_balances
GROUP BY  month, customer_id
),

option1 AS (
SELECT month,
       SUM(running_balance) AS data_required
FROM ranking_closing_bal
GROUP BY month
),

option2 AS (
SELECT month,
       SUM(avg_bal) AS data_required
FROM running_bal_stats
GROUP BY month
),


option3 AS (
SELECT month,
       SUM(max_bal) AS data_required
FROM running_bal_stats
GROUP BY month
)

SELECT *, 'Option 1-Monthly closing balance' AS option_type
FROM option1
UNION ALL
SELECT *, 'Option 2-average balance'
FROM option2
UNION ALL
SELECT *, 'Option 3-maximum balance'
FROM option3
ORDER BY option_type, FIELD(month, 'January','February', 'March', 'April');
```

This challenge is to know how much data storage the Data Bank would need each month, depending on how data allocation is calculated. After calculating the running balance in the CTE running_balances and ranking them in the CTE ranking_closing_balance, its average, minimum, and maximum are calculated aggregated by month and customer_id.

Option 1 – SUM of the previous month's ending balance

Option 2 – SUM of the average balance of that month

Option 3 – SUM of maximum balance (for real-time allocation)

Output:

Month	Data required	Option Type
January	126091	Option 1-Monthly closing balance
February	-34350	Option 1-Monthly closing balance
March	-194916	Option 1-Monthly closing balance
April	-180855	Option 1-Monthly closing balance
January	182333	Option 2-average balance
February	22342	Option 2-average balance
March	-125311	Option 2-average balance
April	-164046	Option 2-average balance
January	351521	Option 3-maximum balance
February	230473	Option 3-maximum balance
March	109155	Option 3-maximum balance
April	-78991	Option 3-maximum balance

Extra Challenge

Data Bank wants to try another option, which is a bit more difficult to implement - they want to calculate data growth using an interest calculation, just like in a traditional savings account you might have with a bank.

If the annual interest rate is set at 6% and the Data Bank team wants to reward its customers by increasing their data allocation based on the interest calculated on a daily basis at the end of each day, how much data would be required for this option every month?

```sql
WITH running_balances AS (
SELECT customer_id,
       txn_date,
       MONTHNAME(txn_date) AS month,
       SUM(CASE WHEN txn_type = "deposit" THEN txn_amount ELSE -txn_amount END)
           OVER(PARTITION BY customer_id
                ORDER BY txn_date) AS balance
FROM customer_transactions
),
```

CTE for calculating running totals for very transaction.

```sql
daily_simple_interest AS (
SELECT customer_id,
       txn_date,
       DATE_FORMAT(txn_date, '%Y-%m') AS month,
       balance,
       ROUND(balance * (0.06/365),2) AS daily_interest
FROM running_balances
),
```

CTE for calculating daily simple interest.

Formula: principle * (interest %/365 days)

```sql
next_transaction AS (
SELECT customer_id,
       txn_date,
       balance,
       LEAD(txn_date) OVER(PARTITION BY customer_id
                           ORDER BY txn_date
                           ) AS next_txn_date
FROM running_balances
),
```

CTE next_transaction is for creating a new column next_tnx_date using the LEAD() function.

CTE activity replaces the NULL values in the column next_txn_date with '2020-12-31' and to find the difference between these dates.

```sql
activity AS (
SELECT customer_id,
       txn_date,
       balance,
       DATEDIFF(COALESCE(next_txn_date, '2020-12-31'), txn_date) AS number_of_days_active_for
FROM next_transaction
),
```

```sql
compounding AS (
SELECT customer_id,
       txn_date,
       DATE_FORMAT(txn_date, '%Y-%m') AS month,
       balance,
       ROUND((balance * POW(1 + 0.06/365, number_of_days_active_for))- balance,2) AS compound_interest,
       ROUND(balance * POW(1 + 0.06/365, number_of_days_active_for),2) AS balance_with_ci
FROM activity
)
```

```sql
SELECT c.month,
       ROUND(SUM(dsi.daily_interest),2) AS total_interest,
       ROUND(SUM(c.compound_interest),2) AS total_compound_interest,
       ROUND(SUM(c.balance_with_ci),2) AS required_data
FROM compounding c
LEFT JOIN daily_simple_interest dsi
     ON c.customer_id = dsi.customer_id
GROUP BY month
ORDER BY month
```

Final query

Compounding CTE: compound interest and balance with compound interest is calculated with the formulas

- **Compound Interest (CI):** $P\left(1+\frac{R}{100}\right)^{T}-P$
- **Compound Amount (A):** $P\left(1+\frac{R}{100}\right)^{T}$

Final query: The final query draws the total of daily simple interest, compounded interest, and balance with compound interest, which are aggregated by month.

Output:

month	total_interest	total_compound_interest	required_data
2020-01	-1078.39	6603.41	4577648.41
2020-02	-850.58	-671.22	-1547872.22
2020-03	-1382.40	-69971.58	-17056158.58
2020-04	-511.14	-149539.66	-9435785.66

Apart from January, every month has a negative balance, which means there were more purchases & withdrawals than deposits.

So, for January, allocate the required data, but for the other months, allocate 0 data. Because withdrawals exceeded deposits overall, data allocation should not be negative & it should be only positive balances.

Conclusion: This case study explains how to use SQL to analyse a digital banking system's customer nodes, transactions, and data allocation. The challenging part of this case study is testing out three different hypothesis options and their calculations. Another challenging part is calculating the simple interest and compound interest and aggregating by each month.

Overall, this case study improves the efficiency of the Data Bank's operations and its planning in data-driven decision-making.

Cheers!

Rohith